1. **review example**

```
#include <stdio.h>
/* not allowed: void array_in (int array_in[][rows], int rows, int columns)*/
void array_in (int rows, int columns, int array_in[][columns])
{
        for(int row = 0; row < rows; row++)
        {
                for(int column = 0; column < columns; column++)
                {
                        printf("a[%d][%d] = %d ", row, column, array_in[row][column]);
                }
                printf("\n");
        }
}

int main()
{
        int rows = 2;
        int columns = 3;
        int array_out[rows][columns];
        for(int row = 0; row < rows; row++)
        {
                for(int column = 0; column < columns; column++)
                {
                        array_out[row][column] = row + column;
                }
        }
        /*
        not allowed:
           int array_out[rows][columns] = {{1, 2, 3},{4, 5, 6}}
           or
           array_in(array_out, rows, columns);
        */
        array_in(rows, columns, array_out);
        return 0;
}
a[0][0] = 0 a[0][1] = 1 a[0][2] = 2
a[1][0] = 1 a[1][1] = 2 a[1][2] = 3
```

2. *Why must the number of columns in two-dimensional arrays be specified ?*

```
| 1 2 2 |                0 1 2 3 4 5 6 7 8
| 9 5 7 | is stored as |1 2 2 9 5 7 8 3 2|.
| 8 3 2 |
```

Computer memory is one-dimensional, so to calculate the exact physical memory address of any given element compiler requires the column index.
address = base address + (row index * number of columns + column index) x data type size
To access element `[1][2]`, where the value is **7**, compiler counts address of the element as 1 * 3 + 2 = 5. Here, **base address** is the address of `array[0][0]` in memory and **data type size** is the memory size in bytes occupied by a single element of the array (for example, 4 bytes for a standard integer, 8 bytes for a double).

3. Understanding the Hanoi Towers
4. Induction and Recursion (gmail.com account)